

Data Analysis with Python 2024–2025

Parte 6: Machine Learning Types

Soluções dos Exercícios

Tradução e Adaptação: Universidade de Helsinque (MOOC.fi)

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**. As soluções apresentadas a seguir foram elaboradas para fins didáticos, seguindo os enunciados dos exercícios propostos no curso original.

Conteúdo

1 Soluções - Capítulo 6 (Clustering)

Exercício 6.5 – Clustering de Plantas

Solução proposta

```
import scipy
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score

def find_permutation(n_clusters, real_labels, labels):
    permutation = []
    for i in range(n_clusters):
        idx = labels == i
        # Escolhe o rotulo mais frequente mapeado no escopo do
        # cluster
        new_label = scipy.stats.mode(real_labels[idx])[0]
        # Validacao cross-version pandas/scipy
        if type(new_label) == np.ndarray:
            new_label = new_label[0]
        permutation.append(new_label)
    return permutation

def plant_clustering():
    iris = load_iris()
    X = iris.data
    y = iris.target

    # Modelo preve em 3 categorias independentes
    model = KMeans(n_clusters=3, random_state=0)
    model.fit(X)

    # Realinha a resposta com a chave e submete ao validador
    permutation = find_permutation(3, y, model.labels_)
    new_labels = [permutation[label] for label in model.labels_]

    return accuracy_score(y, new_labels)
```

Exercício 6.6 – Clusters Não Convexos

Solução proposta

```
import pandas as pd
import numpy as np
import scipy
from sklearn.cluster import DBSCAN
from sklearn.metrics import accuracy_score

def find_permutation_with_outliers(n_clusters, real_labels, labels):
    :
    permutation = []
    for i in range(n_clusters):
        idx = labels == i
        if len(real_labels[idx]) == 0:
            permutation.append(0) # Padrao caso nao haja registros
            continue
        new_label = scipy.stats.mode(real_labels[idx])[0]
        if type(new_label) == np.ndarray:
            new_label = new_label[0]
        permutation.append(new_label)
    return permutation

def nonconvex_clusters():
    df = pd.read_csv("src/data.tsv", sep='\t')
    X = df[['X1', 'X2']].values
    y = df['y'].values

    results = []
    for eps in np.arange(0.05, 0.20, 0.05):
        model = DBSCAN(eps=eps)
        model.fit(X)

        # Filtra quantidade e outliers (-1)
        unique_labels = set(model.labels_)
        num_clusters = len(unique_labels) - (1 if -1 in unique_labels else 0)
        outliers = list(model.labels_).count(-1)

        # Verificando se os espacos convergem com os reais previstos (2 clusters isolados)
        if num_clusters == len(set(y)):
            perm = find_permutation_with_outliers(num_clusters, y, model.labels_)
            mask = model.labels_ != -1
            acc = accuracy_score(y[mask], [perm[label] for label in model.labels_[mask]])
        else:
            acc = np.nan

        results.append([eps, acc, num_clusters, outliers])

    return pd.DataFrame(results, columns=["eps", "Score", "Clusters", "Outliers"])
```

Exercício 6.7 – Locais de Ligação Molecular

Solução proposta

```
import pandas as pd
import numpy as np
import scipy
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import accuracy_score, pairwise_distances

def toint(nucleotide):
    mapping = {'A': 0, 'C': 1, 'G': 2, 'T': 3}
    return mapping.get(nucleotide, -1)

def get_features_and_labels(filename):
    df = pd.read_csv(filename, sep='\t')

    # List comprehension encadeada gerando vetores inteiros
    X = np.array([[toint(char) for char in seq] for seq in df['X']
                  ])
    y = df['y'].values
    return X, y

def find_permutation(n_clusters, real_labels, labels):
    permutation = []
    for i in range(n_clusters):
        idx = labels == i
        new_label = scipy.stats.mode(real_labels[idx])[0]
        if type(new_label) == np.ndarray:
            new_label = new_label[0]
        permutation.append(new_label)
    return permutation

def cluster_euclidean(filename):
    X, y = get_features_and_labels(filename)
    # Classificador aglomerativo Euclidiano
    model = AgglomerativeClustering(n_clusters=2, linkage='average',
                                     affinity='euclidean')
    labels = model.fit_predict(X)

    perm = find_permutation(2, y, labels)
    new_labels = [perm[l] for l in labels]
    return accuracy_score(y, new_labels)

def cluster_hamming(filename):
    X, y = get_features_and_labels(filename)

    # Processa os componentes metricos usando algoritmos focados no
    # modelo Hamming
    dist_matrix = pairwise_distances(X, metric='hamming')

    # Utiliza as distancias calculadas previamente via precomputed
    model = AgglomerativeClustering(n_clusters=2, linkage='average',
                                     affinity='precomputed')
    labels = model.fit_predict(dist_matrix)

    perm = find_permutation(2, y, labels)
    new_labels = [perm[l] for l in labels]
    return accuracy_score(y, new_labels)
```